

Week 5

DevOps Internship

Release Date: 24th August 2026

This week, you'll learn **Jenkins, CI/CD & GitHub Integration**. CI/CD is a core DevOps practice that automates software delivery, while Jenkins helps teams build, test, and validate applications continuously.

Topic:- Jenkins, CI/CD & GitHub Integration

- Jenkins Fundamentals & Architecture
- Jenkins Installation & Configuration
- Jenkins Jobs & Builds
- Jenkins Credentials
- GitHub Integration
- Freestyle Projects
- Declarative Pipelines
- Jenkinsfile
- Pipeline Stages
- Build Triggers
- GitHub Webhooks
- Build & Test Automation
- Pipeline Troubleshooting
- CI/CD Best Practices

*Learning Approach

This week, focus on understanding **how code moves through a CI/CD pipeline** rather than simply creating Jenkins jobs. As you progress, start thinking about how automation and AI can work together to make DevOps workflows faster and smarter.

Keep these points in mind:

- ✓ Understand every stage of your Jenkins pipeline.
- ✓ Write and modify your own Jenkinsfile.
- ✓ Learn how GitHub Webhooks trigger automated builds.
- ✓ Read Jenkins Console Output when a build fails.
- ✓ Use AI as an assistant for troubleshooting, optimization, and automation—not as a replacement for understanding.

*Complete the following practical exercises

- Install and configure Jenkins.
- Explore the Jenkins Dashboard.
- Connect Jenkins with GitHub.
- Configure Jenkins Credentials securely.
- Create a **Freestyle Project** that pulls code from GitHub.
- Configure an automated build trigger.
- Create a **Declarative Pipeline**.
- Write a Jenkinsfile.
- Create pipeline stages for:
 - Checkout
 - Build
 - Test
- Execute the pipeline and analyze the build logs.
- Configure basic GitHub Webhook integration.
- Prepare a short PDF explaining:
 - CI/CD
 - Jenkins Architecture
 - Freestyle Jobs
 - Declarative Pipelines
 - Jenkinsfile
 - GitHub Webhooks
 - Jenkins Credentials
 - Pipeline Stages

Task Submission

Once you complete the task, submit:

- ✓ GitHub Repository Link
- ✓ PDF Report
- ✓ Screenshot of Jenkins Dashboard
- ✓ Screenshot of GitHub Integration
- ✓ Screenshot of Pipeline Execution
- ✓ Screenshot of Build Logs

Task Submission Deadline: Wednesday (EOD)

Scenario

Imagine you are working as a **Junior DevOps Engineer** in a development team. Developers are continuously pushing code to GitHub, and your responsibility is to create a CI pipeline that automatically validates every change.

Complete the following:

- Create a GitHub repository for the application.
- Connect the repository with Jenkins.
- Configure a GitHub Webhook.
- Create a Jenkins Pipeline with separate stages:
 - **Checkout**
 - **Build**
 - **Test**
 - **Validation**
- Push a new change to GitHub.
- Verify that Jenkins automatically starts the pipeline.
- Introduce a controlled error into the project.
- Observe the failed Jenkins build.
- Analyze the Console Output to identify the issue.
- Fix the error and run the pipeline again.
- Document the complete failure → troubleshooting → resolution process.

Activity Submission

After completing the hands-on activity, submit:

- ✓ GitHub Repository Link
- ✓ Screenshot of GitHub Webhook
- ✓ Screenshot of Successful Pipeline
- ✓ Screenshot of Failed Pipeline
- ✓ Screenshot of Console Output
- ✓ Screenshot of Additional Validation Stage
- ✓ Short troubleshooting summary

Activity Submission Deadline: Friday (EOD)

Use an AI assistant such as **ChatGPT, GitHub Copilot, Amazon Q, Gemini, Claude, or Microsoft Copilot** while working on your Jenkins pipeline.

Try the following:

- Ask AI to explain your Jenkinsfile **stage by stage**.
- Ask AI to review your pipeline and identify possible improvements.
- Give AI a Jenkins build error from your Console Output and ask it to identify the likely cause.
- Ask AI to suggest a better pipeline structure for your project.
- Ask AI to generate a basic test stage for your application.
- Ask AI to suggest how you can add a **security or code-quality validation stage**.
- Compare the AI's suggestion with your own solution and decide what should actually be implemented.

***No separate AI Task submission is required.**

Take your Jenkins pipeline and ask:

"How can AI be integrated into this CI/CD pipeline to improve code validation, troubleshooting, testing, or developer feedback?"

From the suggestions, choose **one practical idea** and try implementing it or creating a working proof of concept.

Important: Don't blindly copy AI-generated Jenkinsfiles or commands. Understand, test, and verify everything before adding it to your pipeline.

***No separate AI Task submission is required.**

The purpose of this activity is to make AI a practical part of your DevOps workflow.

DevOps Pro Tip

A professional CI/CD pipeline should fail safely and provide useful feedback. When a build fails, don't immediately rerun it read the logs, identify the root cause, fix the issue, and then verify the pipeline again.

***Understand how Jenkins moves code through each stage rather than simply making the pipeline turn green. This week is about developing the mindset of a DevOps Engineer who can automate, troubleshoot, and improve delivery workflows.**

***If you face any genuine issues, feel free to reach out before the submission deadline.**